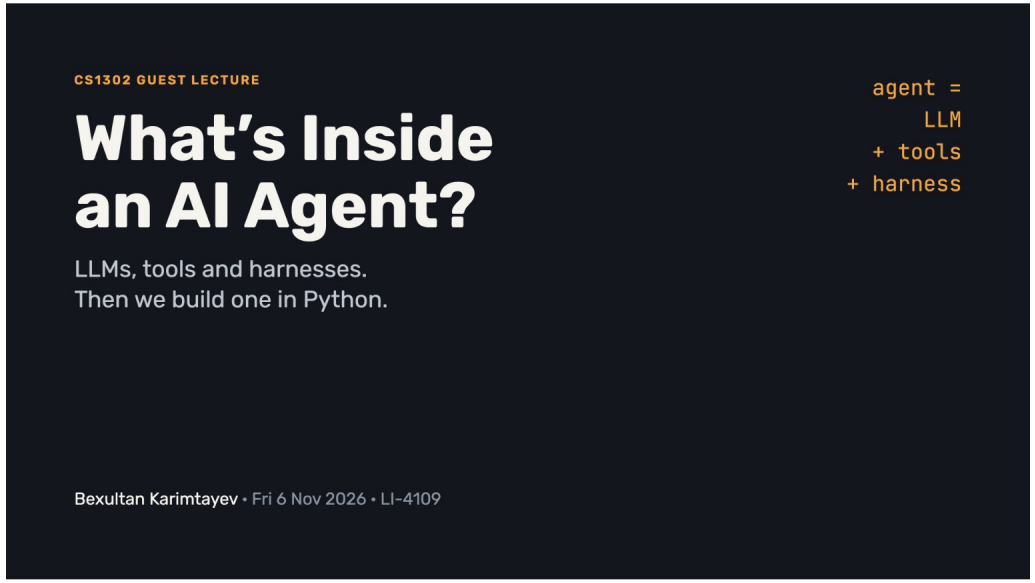
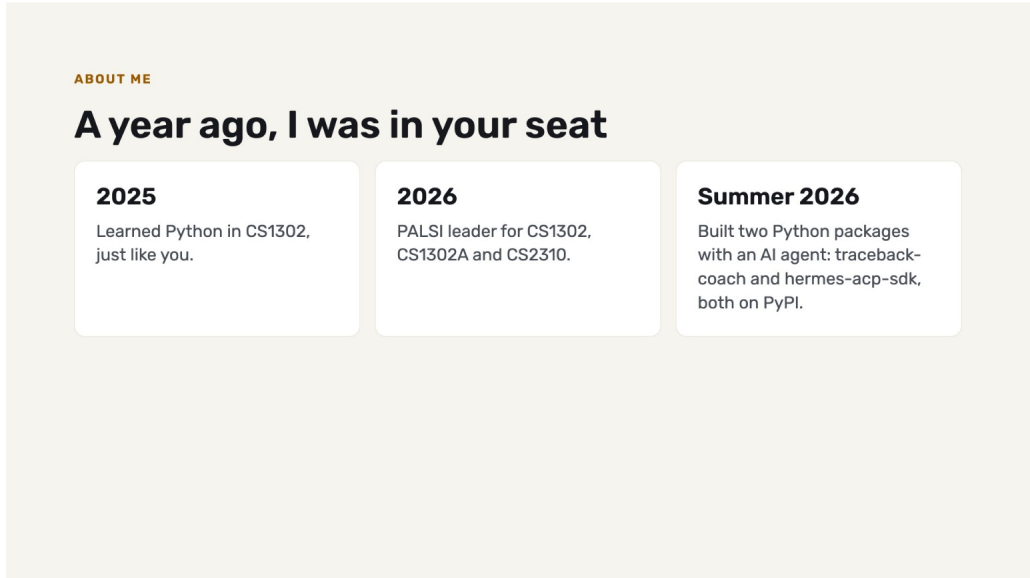


[~0:30 • Why now: make a promise] Hi everyone, I'm Bexultan. By the end of today you'll know what is inside an AI agent like Hermes, and we'll build a small one together, using only Python from CS1302.



[~0:45 • Why now: you are like them] A year ago I was learning Python in this course, exactly where you are now. This year I help run PALSI for CS1302, CS1302A and CS2310, and over the summer I built two Python packages with an AI agent. I'm not an AI researcher: everything today uses Python you already know.



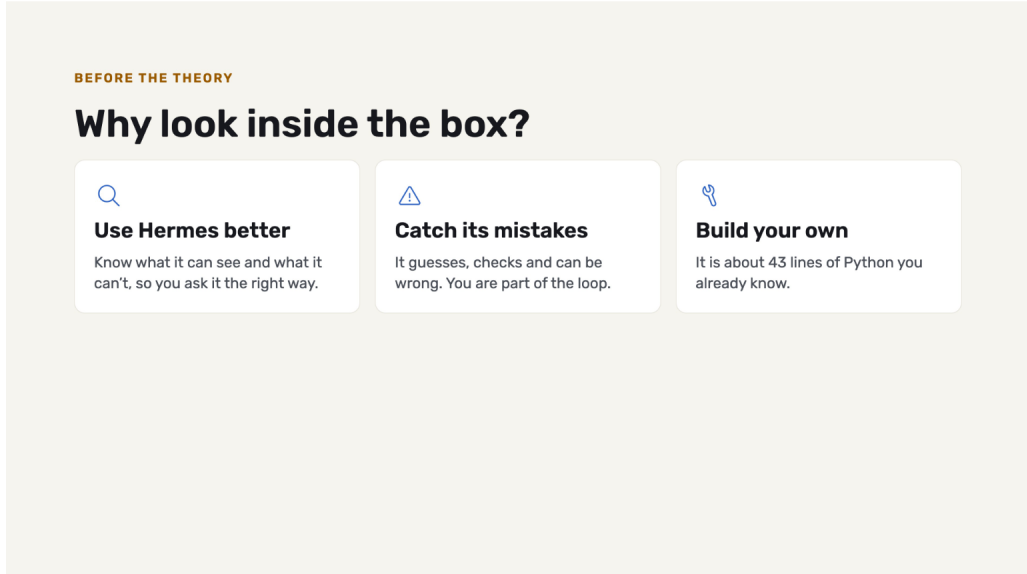
[~0:30 • Why now: start from what they use every week] You have all used Hermes in DIVE. You type a request, and it writes code, runs it, sees an error and fixes it. It feels like magic. Let me show you one I built.



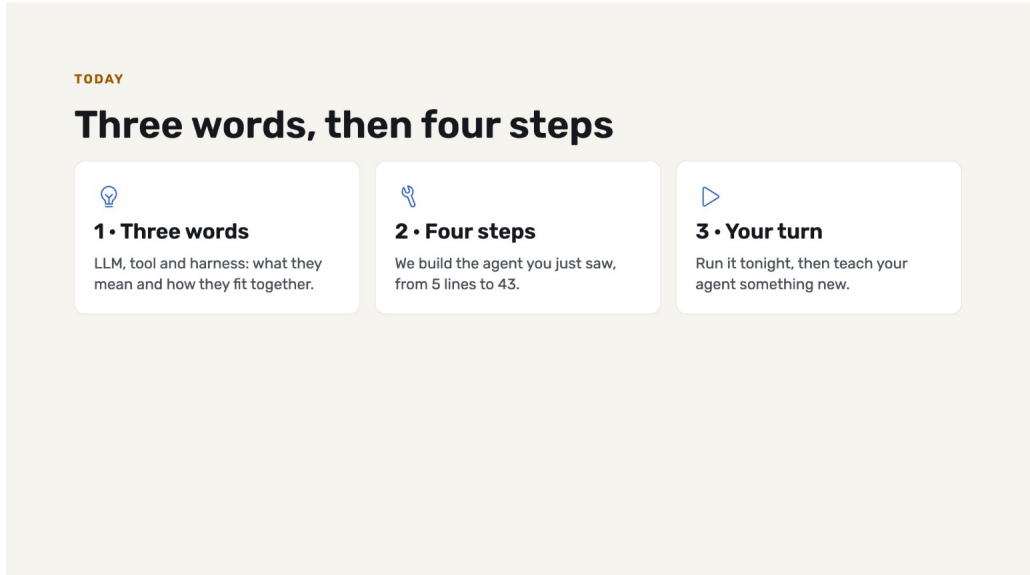
[~2:00 • Why now: show the goal before any theory, so every later idea answers ‘how did it do that?’] Before the talk: python check_llm.py must say Ready for the demo. Never open .env on screen. Run: python mini_agent.py. Task: Write is_prime in primes.py and test it. Point at the [step] lines, the y/n question and the [result] lines. FALLBACKS: API or Wi-Fi fails → run python offline_demo.py (same harness, recorded LLM replies). Laptop fails → go to the next slide and say ‘here is a run I recorded earlier’.



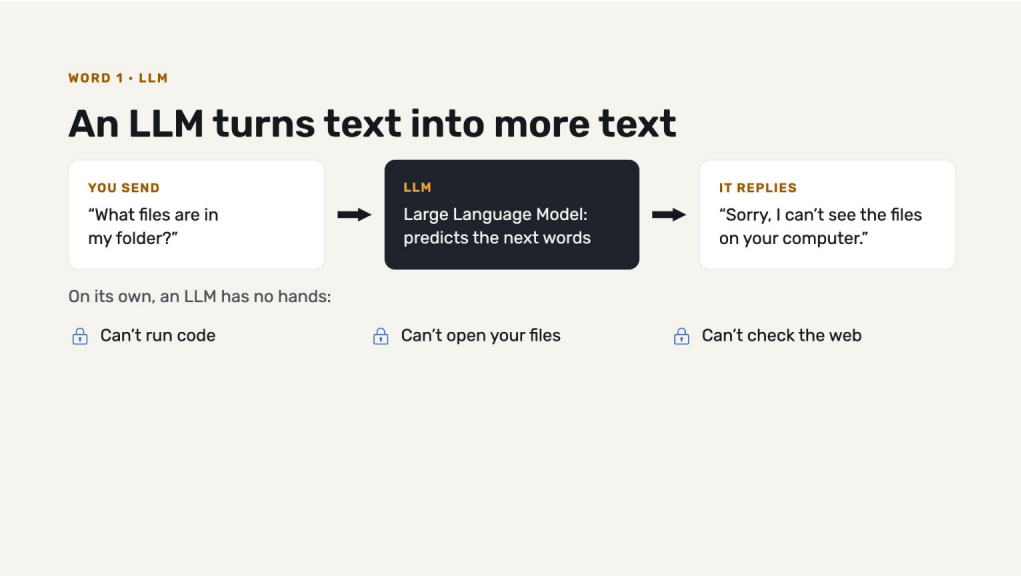
[~1:00 • Why now: freeze the run so everyone sees the key moment] If the live demo worked, go through this quickly: it is the same run. If it failed, this IS the demo: ‘here is a run I recorded earlier’. Step 1, the LLM asks to save a file. Step 2, it asks to run a test, and I say yes. The result is wrong: 0 and 1 are not prime. Nobody told it, but in step 3 it fixes the code and in step 4 the test passes. Then ask the three questions on the slide and leave them open: who wrote the code, who ran it, how did it notice the bug? We’ll answer all three today.



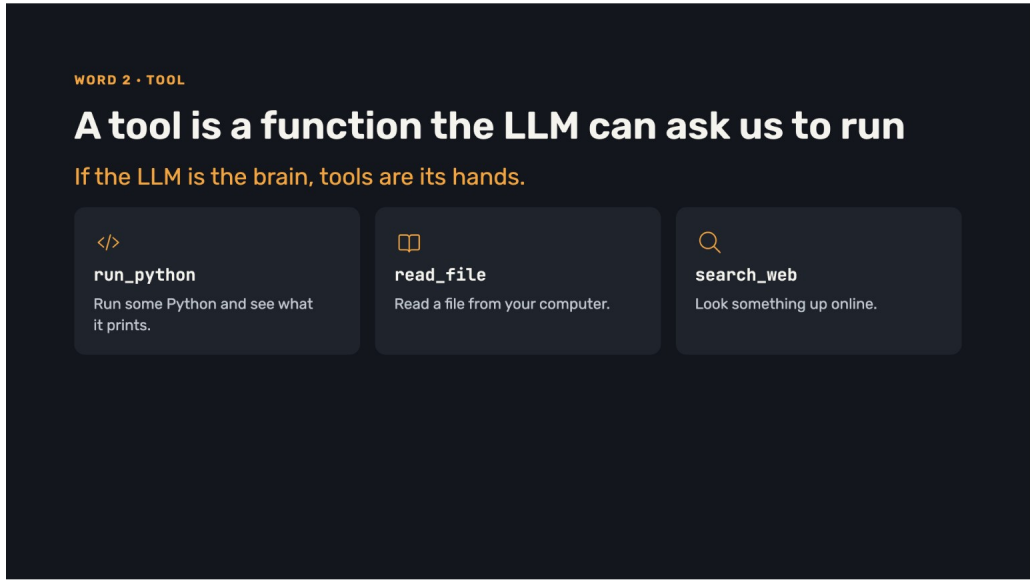
[~0:45 • Why now: before the theory, give them a reason to care] Why should you care what is inside? Three reasons. You'll use Hermes better, because you'll know what it can and can't see. You'll catch its mistakes, because you'll know it guesses and checks, and you are part of that loop. And you can build your own: it's about 43 lines of Python. To answer our three questions we only need three words.



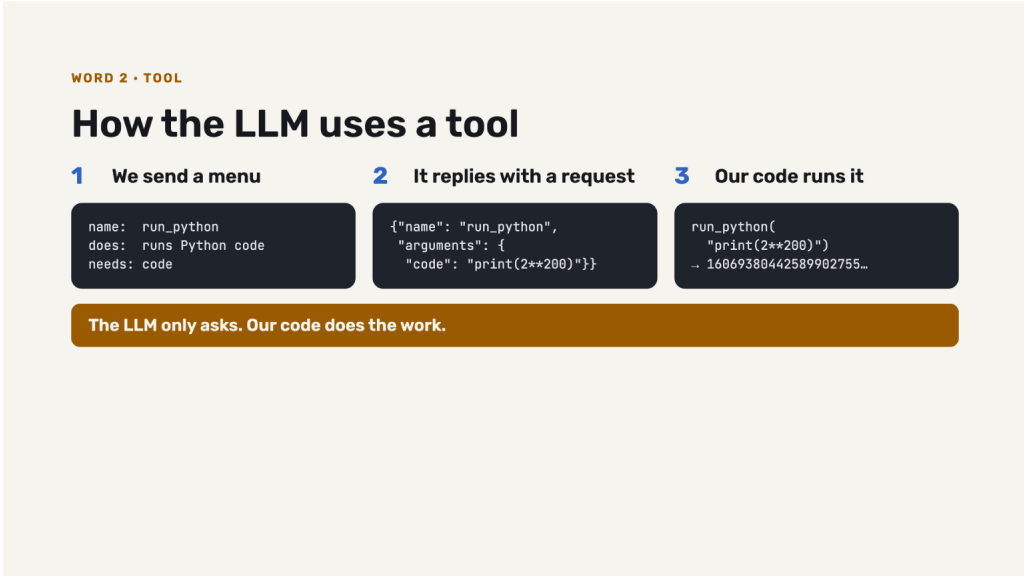
[~0:20 • Why now: show the route] The plan: three words, LLM, tool and harness. Then we build the agent you just saw in four small steps, each a file you can run. Then it is your turn.



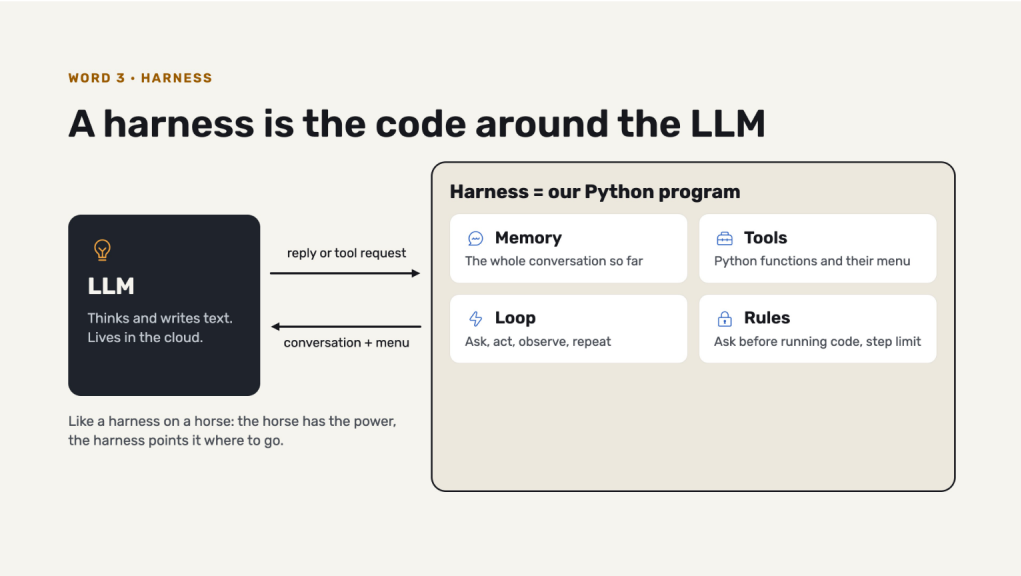
[~1:30 • Why now: start from what they know, ChatGPT, and show its limit] Word one: LLM, a large language model, the brain inside ChatGPT or Hermes. It does one thing: text in, more text out. Ask it what files are in your folder and it can only guess or apologise. It can’t run code, open your files or check the web. But our agent just ran code! So something must give it hands.



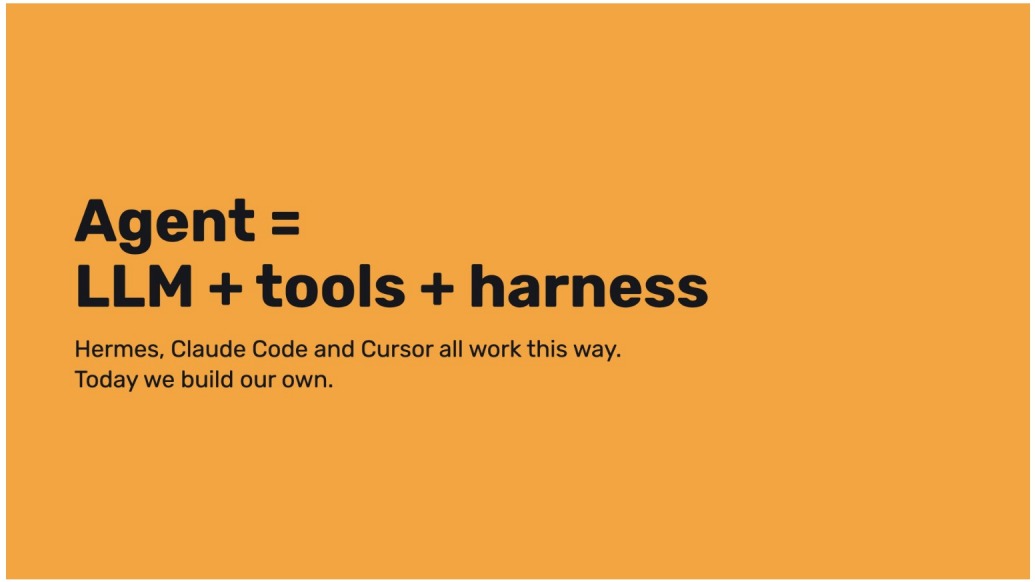
[~1:00 • Why now: answers ‘how did it run code?’] Those hands are called tools. A tool is just a Python function that we allow the LLM to use: run_python, read_file, search_web. But wait: the LLM lives in the cloud and the function is on my laptop. How can it use it?



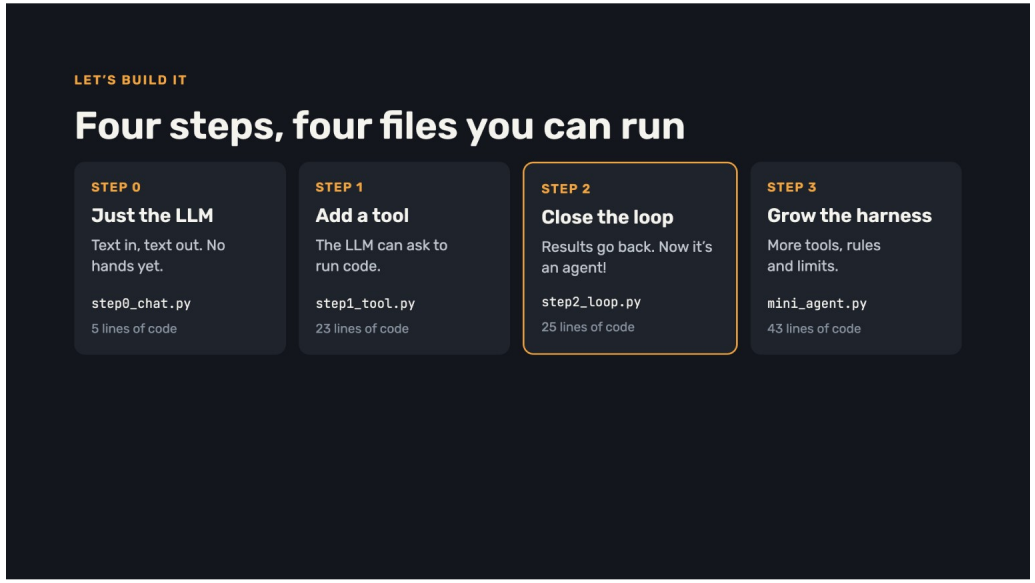
[~1:30 • Why now: explain the mechanism, answers ‘who ran the code?’] Three steps. One: with every message we send a menu of tools: name, what it does, what it needs. Two: instead of text, the LLM can reply with an order from the menu: please run this code. Three: our Python code runs it. So the answer to ‘who ran the code?’ is: our code. The LLM only asks. And that code has a name.



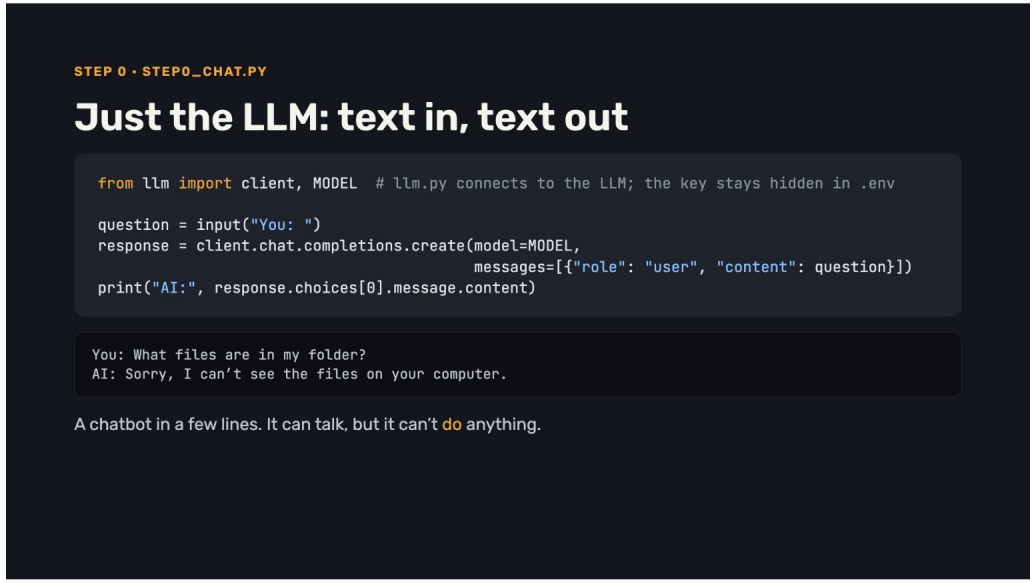
[~2:00 • Why now: name ‘our code’, answers ‘how did it notice the bug?’] It is called the harness: the program around the LLM. It keeps the memory, the whole conversation. It holds the tools and their menu. It runs the loop. It enforces rules, like asking before running code. Because the harness puts every result back into the conversation, the LLM saw the wrong output: that is how it noticed the bug. The name comes from a horse’s harness: the horse has the power, the harness points it somewhere useful.



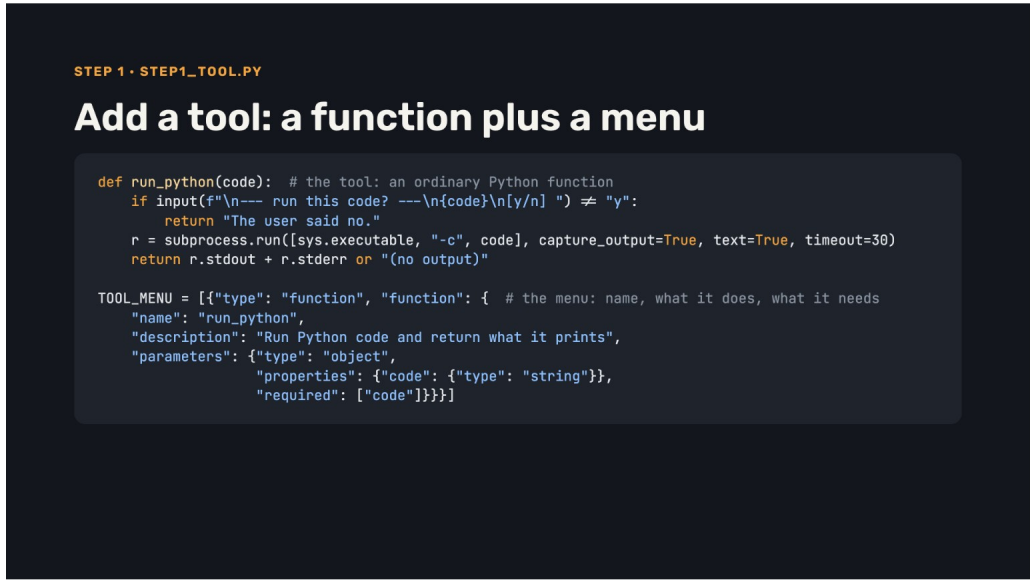
[~0:20 • Why now: one sentence to remember before building] Agent equals LLM plus tools plus harness. Hermes, Claude Code, Cursor: same three parts. The LLM we rent. Tools and harness we can write ourselves. Let’s do it.



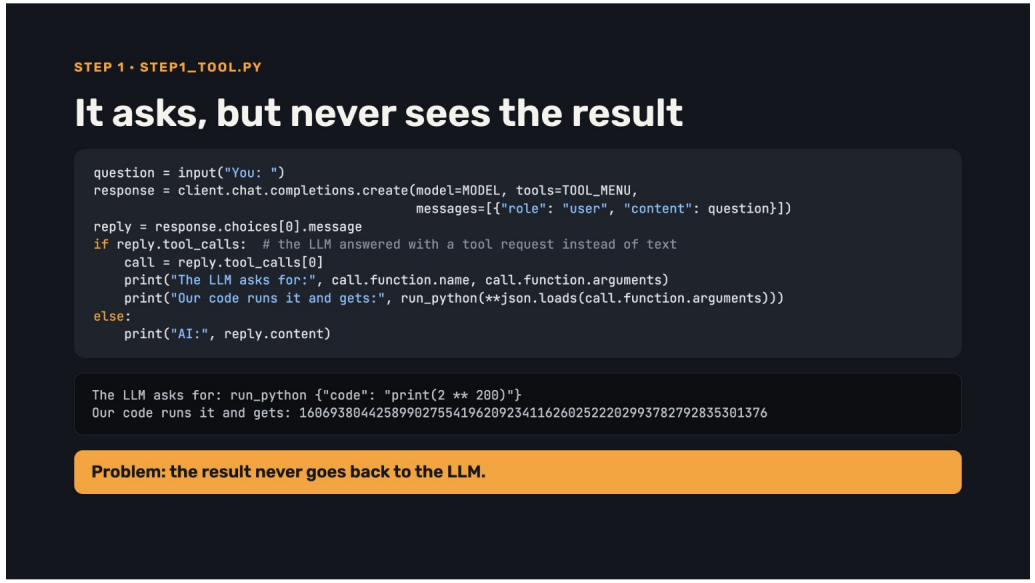
[~0:30 • Why now: show where each step leads] Four steps, four files you can run tonight. Each step fixes one problem from the step before. Step 2 is the moment it becomes an agent.



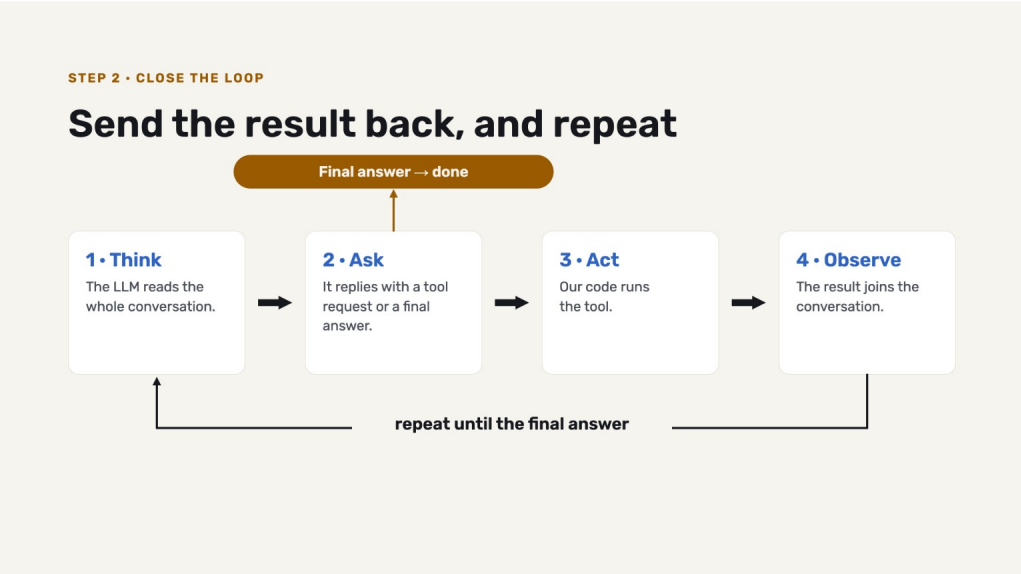
[~1:00 • Why now: the smallest working program, so they see the starting point] The first line imports the connection from llm.py: one small file that talks to CityU's DeepSeek server, and keeps my key hidden in a private file. Then a complete chatbot: read a question, send it, print the answer. Ask about your files and it apologises. It can talk, but it can't do anything. Problem: no hands. So, step 1: give it a tool.



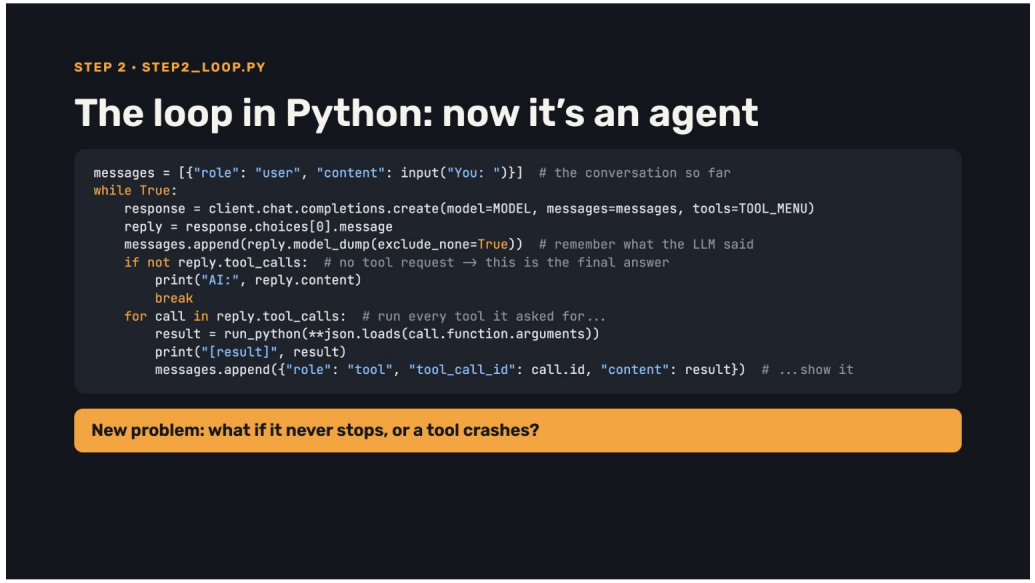
[~1:30 • Why now: hands = tools, built from things they know] Two new things. The tool: run_python, an ordinary function. It asks a human y or n before running anything, our first safety rule. And the menu: a dictionary with the name, a plain-English description and the parameters. The LLM never sees our Python code, only this menu.



[~1:00 • Why now: show it works, then the problem that motivates the loop] We send the question with the menu, and the reply is a tool request. Our code runs it and gets the exact answer. But look: we printed the result and the program ended. The LLM never saw it. It can’t check its work or fix mistakes. We need to send the result back.



[~1:00 • Why now: the fix for that problem is a loop] Here's the fix. One: the LLM reads the whole conversation. Two: it replies with a tool request or a final answer. If final, we stop. Three: our code runs the tool. Four: the result joins the conversation, and we go round again. This loop is the heart of every agent.



[~1:30 • Why now: the loop in code, the moment it becomes an agent] messages is the conversation, a list of dictionaries. Each time round we send it with the menu. No tool calls: print the answer and break. Otherwise run each tool and append the result, so the LLM sees it next time. About a dozen lines, and it is an agent. New problem: while True. What if it never stops? What if a tool crashes?



[~1:30 • Why now: fix the new problems by growing the harness] Two more tools, write_file and read_file. A step limit of ten. try and except, so a failing tool sends the error back to the LLM instead of crashing, and it can fix its mistake. A timeout for code that never ends. The y/n question stays. And a system prompt: standing orders like ‘test your code’. 43 lines, and that is the agent from the demo.

EXAMPLE RUN · OFFLINE_DEMO.PY

One task, step by step

Step	The LLM asks for	Our code sends back
1	write_file: save primes.py	Wrote 107 characters to primes.py
2	run_python: test is_prime	[0, 1, 2, 3, 5, 7, 11] (wrong: 0 and 1 are not prime)
3	write_file: save a fixed primes.py	Wrote 142 characters to primes.py
4	run_python: test again	[2, 3, 5, 7, 11] (correct)
5	No tool: final answer	"Done. My first version said 0 and 1 are prime…"

Nobody told it about the bug. It read the test output in the conversation and fixed it.

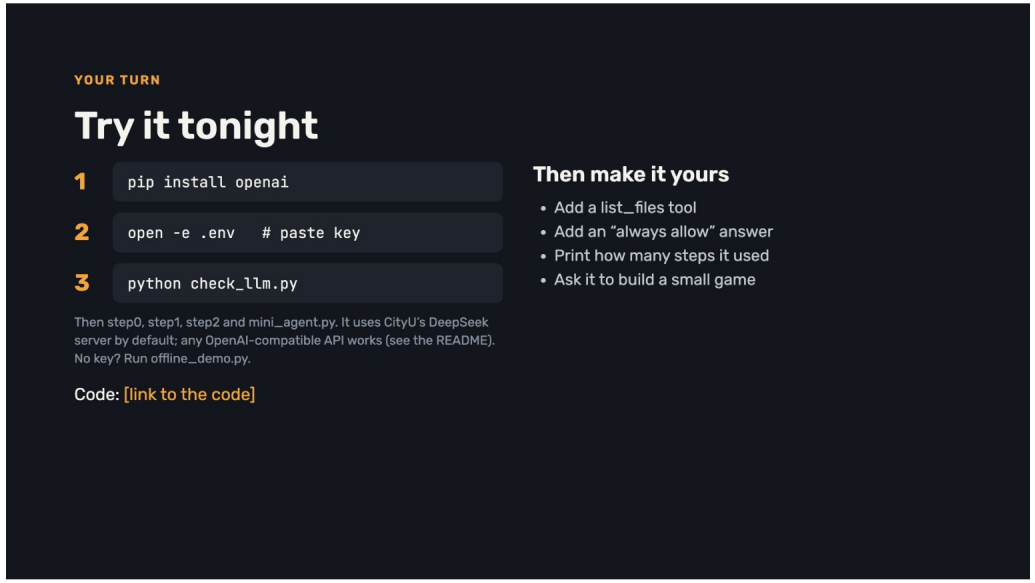
[~1:00 • Why now: close the circle, the demo is no longer magic] Remember the demo? Now you can read every line. Steps 1 and 3: tool requests. Step 2: the harness ran the test and put the wrong output into the conversation. That is how it noticed the bug. Step 5: no tool request, so the loop ended.

FROM 43 LINES TO A REAL AGENT

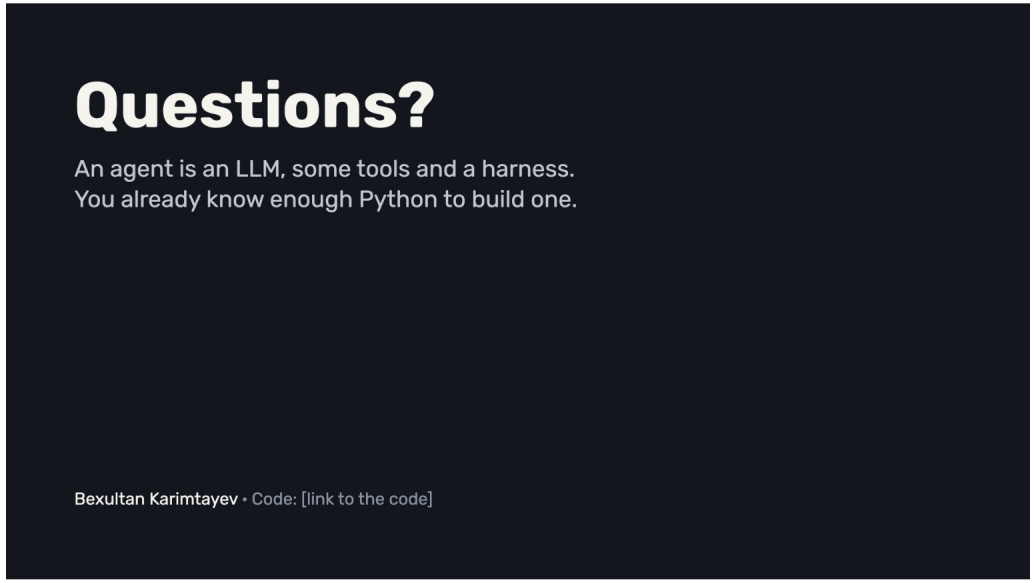
Real harnesses are bigger, but the loop is the same

	Our harness	Real coding agents
Tools	3	Dozens: edit, search, terminal, web
Memory	One task	Remembers across sessions
Safety	A y/n question	Sandboxes and permission rules
Long tasks	Up to 10 steps	Plans, summarises and keeps going

[~1:00 • Why now: connect back to what they use] So what do Hermes or Claude Code add? More tools, memory between sessions, sandboxes so code can’t damage your computer, tricks for long tasks. The harness is much bigger. But open any of them up and you find the same loop.



[~0:45 • Why now: turn understanding into action tonight] Install openai, paste your key into the .env file, and run check_llm.py to test the connection. Then run the steps in order. Any OpenAI-compatible API works; the README explains how to switch. No key? offline_demo.py still runs. Then make it yours: add a tool, count the steps, ask it to build a game.



[Q&A • aim to reach this slide by about minute 23] One thing to remember: an agent is an LLM, some tools and a harness, and the harness is a loop in Python. You already know enough to build one. Questions?